Instituto Superior de Engenharia de Lisboa (ISEL)
Departamento de Engenharia Eletrónica e de Telecomunicações e Computadores (DEETC)

LEIM
Licenciatura em Engenharia Informática e Multimédia
Unidade Curricular de Projeto

Sistema de Resumos de Processos Crime



Gonçalo Barbosa (49845)

Marta Ferreira (50775)

Orientadores

Professor Hélder Bastos

Inspetor Chefe Francisco Luís

Junho, 2025

Resumo

Resumir processos crime é uma tarefa lenta e por vezes repetitiva. Os métodos atuais envolvem a exigência de leitura, interpretação jurídica, tempo e mão de obra especializada.

O sistema proposto utiliza inteligência artificial para gerar relatórios finais de forma automática e rápida quando comparada aos métodos anteriores.

Através de exemplos e alimentado por documentos de processos-crime reais, o sistema cria sínteses claras e objetivas focadas nos relatórios criminais.

Palavras-chave: processos crime, inteligência artificial, relatórios jurídicos, sumarização de documentos, automatização.

Abstract

Summarizing criminal cases is a slow and often repetitive task. Current methods require legal reading, interpretation, time, and specialized human resources.

The proposed system uses artificial intelligence to automatically and rapidly generate final reports, offering a significant improvement over traditional methods.

Trained on examples and real criminal case documents, the system produces clear and objective summaries focused on criminal reports.

Keywords: criminal cases, artificial intelligence, legal reports, document summarization, automation.

Agradecimentos

Gostaríamos, inicialmente, de agradecer ao Professor Hélder Bastos, pela proposta cativante e por toda a mentoria e auxílio durante o desenvolvimento do projeto.

De seguida, também queríamos expressar a nossa gratidão ao Professor Doutor Nuno Datia por ter concordado ser o arguente do nosso projeto e pelas recomendações que nos foram concedidas.

Agradecemos profundamente ao Inspetor Chefe Franscisco Luís e à organização da Polícia Judiciária, particularmente à Unidade Nacional de Combate ao Cibercrime e Criminalidade Tecnológica por nos terem disponibilizado um espaço para o nosso crescimento autónomo e para o progresso da proposta de projeto.

Finalmente, gostaríamos imensamente de agradecer a ambas as nossas famílias por nos terem dado todo o apoio incondicional durante o desenvolvimento desta nova etapa.

Eu, Gonçalo Barbosa, dedico este projeto à minha família, por sempre ter acreditado em mim mesmo nos momentos mais difíceis, e por estarem incondicionalmente sempre presentes ao meu lado, juntamente com a educação que me foi concedida ao longo do meu crescimento académico.

Eu, Marta Ferreira, dedico este projeto à minha família, por me terem apoiado incondicionalmente todos os dias, mesmo quando não acreditava no meu próprio potencial e por estarem sempre presentes e prontos para ajudar.

Ambos, dedicamos este projeto a toda a comunidade que fizemos no ISEL, passando por todos os professores e colegas que nos acompanharam ao longo desta jornada.
A todos vocês, um enorme obrigado.

Índice

Resumo **i**

Abstract **iii**

Agradecimentos **v**

Índice **ix**

Lista de Tabelas **xi**

Lista de Figuras **xiii**

1 Introdução **1**

2 Trabalho Relacionado **3**

2.1 Fundamentação Teórica 3

2.1.1 Processos Crime 3

2.1.2 Inteligência Artificial 3

2.1.3 *Large Language Models* 4

2.1.4 *Ollama* 4

2.1.5 *Prompt Engineering* 4

2.1.6 Tecnologias Utilizadas 6

2.1.7 Trabalho Relacionado 7

3 Modelo Proposto **9**

3.1 Requisitos 9

3.1.1 Requisitos Funcionais 9

3.1.2 Requisitos Não Funcionais 10

3.2 Casos de Utilização 11

3.3 Abordagem 14

4 Implementação do Modelo **17**
4.1 Primeira Interação com o Flask 17
4.2 Desenvolvimento da implementação 18
4.3 Implementação da Solução Desenvolvida 22
4.3.1 Interface Web e Experiência do Utilizador 22
4.3.2 Estrutura Modular e Lógica de Backend 26

5 Validação e Testes **33**
5.1 Geração de Relatórios Finais 33
5.2 Comparação com um Relatório Real 41

6 Conclusões e Trabalho Futuro **43**
6.1 Conclusões 43
6.2 Trabalho Futuro 44

A UML **47**

B Agente ResumoGeral **49**

C Agente ResumoCronologico **53**

Bibliografia **57**

Lista de Tabelas

3.1 Tabela de Requisitos funcionais 10
3.2 Tabela de Requisitos funcionais 10

Lista de Figuras

3.1 Modelo de Casos de Utilização 12

4.1 Página Inicial da Aplicação *Web* 23
4.2 Ficheiros Submetidos pelo Inspetor 23
4.3 Visualização do Ficheiro submetido 24
4.4 Respostas Geradas pelo agente criado 25
4.5 Possível visualização e edição das respostas geradas 25
4.6 Possível visualização, edição e *download* do Relatório Final . . 26

5.1 Comparação entre Relatórios 42

A.1 UML 48

Chapter 1

Introdução

O sistema proposto, denominado de Sistema de Resumos de Processos Crime, é um projeto cujo objetivo é sintetizar processos-crime. O acesso digital aos processos-crime por parte dos inspetores da Polícia Judiciária constitui uma necessidade crescente no contexto da modernização e otimização das atividades de investigação criminal. Este projeto propõe o desenvolvimento de um sistema que, recorrendo à execução de agentes baseados em *Large Language Models*, doravante designados por *LLMs*, permite apoiar de forma eficiente a análise de documentos e a produção automática de relatórios criminais finais, assegurando rigor técnico, coesão e ganhos significativos em termos de produtividade.

O acréscimo da quantidade de processos-crime representa um dos principais desafios operacionais enfrentados pela Polícia Judiciária, sendo esta uma principal causa que exige uma mudança no que toca à gestão de processamento de dados jurídicos. Neste contexto, identificou-se uma necessidade de otimizar a produtividade no trabalho dos inspetores, especificamente nas fases de leitura, interpretação e produção de relatórios criminais.

O projeto envolveu a implementação de uma plataforma web que permite a inserção e gestão digital de processos-crime. Foram concebidos agentes inteligentes executados em *LLMs*, com o intuito de explorar diversas configurações de *system prompts*, cuja formulação e otimização foram ajustadas para obter resultados linguisticamente consistentes, tecnicamente relevantes e adequados ao contexto formal exigido. Para isso, foram também utilizados múltiplos processos já existentes fornecidos para recolher dados relevantes e avaliar o desempenho do sistema.

Para verificar os resultados de forma viável, foi também implementado um mecanismo de validação com um agente que irá determinar se um relatório foi feito por um inspetor ou gerado automaticamente, demonstrando que o sistema é capaz de gerar relatórios com um grau de similaridade elevado aos elaborados manualmente.

Este relatório vai consistir numa estrutura organizada por capítulos, de forma a guiar o leitor, de modo progressivo e fundamentado, através das várias etapas que compõem o desenvolvimento deste projeto. O Capítulo 1 é onde o leitor se encontra no momento, e visa a descrição e a introdução dos conteúdos do desenvolvimento do projeto. No Capítulo 2, são apresentados os principais fundamentos teóricos e tecnológicos que sustentam toda a evolução do projeto, passando por diversos conceitos, tal como processos-crime, *prompt engineering*, *LLMs* e *Generative AI*, pela descrição das linguagens e bibliotecas utilizadas e pelo detalhe da plataforma *Docker*. O Capítulo 3 incide sobre o modelo proposto de desenvolvimento, detalhando a estrutura funcional da aplicação, os seus casos de utilização e respetivos requisitos. No Capítulo 4, é aprofundada uma explicação sobre a implementação do projeto, que passa por todas as componentes desenvolvidas (*frontend* e *backend*), a integração dos agentes *LLM* e os procedimentos de otimização aplicados. Inclui-se ainda uma descrição do *pipeline* de processamento dos dados inseridos na plataforma *web* e das abordagens utilizadas na definição dos *prompts* para os agentes. De seguida, o Capítulo 5 apresenta uma fase de testes, validações e análise crítica de todos os resultados obtidos, conseguindo evidenciar o desempenho positivo dos agentes criados, no que toca ao objetivo do projeto. Finalmente, o Capítulo 6 vai sintetizar as conclusões do projeto desenvolvido e apresentar um conjunto de perspetivas de desenvolvimento futuro, especialmente em termos de escalabilidade e possíveis melhorias do sistema.

Ao longo do documento, adotaram-se as seguintes convenções: itálico para estrangeirismos e "aspas" para citações.

Chapter 2

Trabalho Relacionado

Um dos principais objetivos no desenvolvimento deste projeto é explorar o potencial da inteligência artificial, mais especificamente dos modelos de linguagem de última geração, neste caso *LLMs*, na análise de processos-crime. A ideia principal é conseguir utilizar esses agentes para auxílio na leitura, interpretação e organização da grande quantidade de informação contida em inquéritos e processos investigativos.

Para isso, alguns conceitos têm de ser bem compreendidos com o objetivo de conseguir enquadrar a teoria dos mesmos com a evolução do projeto.

2.1 Fundamentação Teórica

2.1.1 Processos Crime

Um processo-crime é o conjunto de atos processuais destinados à investigação, instrução e julgamento de factos que possam constituir um crime. Trata-se de um procedimento formal conduzido pelas autoridades judiciárias e policiais, com o objetivo de apurar a veracidade dos acontecimentos, identificar os autores e garantir que a justiça penal seja aplicada de forma equitativa [da República, site].

2.1.2 Inteligência Artificial

O conceito de Inteligência Artificial [Russell e Norvig, 2020] refere-se à capacidade de sistemas computacionais executarem diversas tarefas que normalmente exigem inteligência humana. O objetivo desta nova tecnologia visa

o desenvolvimento de máquinas e algoritmos que sejam capazes de aprender com dados reais, tomar decisões, fazer previsões e resolver problemas de forma autónoma. Os sistemas de inteligência artificial visam simular processos cognitivos humanos, como a análise de uma enorme quantidade de informação, identificação de padrões, desempenhando funções complexas de uma forma melhorada e mais rápida.

2.1.3 *Large Language Models*

Os *Large Language Models*, *LLMs*, incidem sobre um tipo avançado de inteligência artificial que, por sua vez, são treinados com enormes quantidades de dados textuais, provenientes de livros, artigos, *websites* e qualquer outro tipo de repositórios públicos. Estes modelos usufruem da aquisição da compreensão, geração e manipulação de uma linguagem natural semelhante à de um ser humano [Naveed et al., 2024].

2.1.4 *Ollama*

Para garantir a execução eficiente e local dos *LLMs*, recorreu-se à ferramenta *Ollama* [Ollama, 2023]. Esta plataforma permite carregar e utilizar modelos de linguagem diretamente num ambiente local, eliminando a necessidade de dependência de serviços externos. Esta abordagem é bastante vantajosa, pois, nenhum processo-crime fica exposto e, por consequência, potencialmente comprometido. Esta ferramenta permite um maior controlo sobre os dados processados e uma maior segurança na manipulação de conteúdos.

O modelo utilizado foi o *Llama* 3.2, sendo que o mesmo foi escolhido tendo em conta as capacidades limitadas das máquinas utilizadas. Este modelo oferece duas versões, mil milhões (1B) ou três mil milhões (3B) de parâmetros *text-in/text-out*, sendo que a última referida foi a integrada no projeto [Ollama, 2024]. A escolha deste modelo deve-se ao seu desempenho superior quando comparado com outras alternativas na mesma faixa de capacidade.

2.1.5 *Prompt Engineering*

No contexto deste projeto, recorreu-se a técnicas de *prompt engineering*, um processo que consiste na elaboração estruturada de instruções fornecidas aos

LLMs, com o objetivo de maximizar a precisão e contextualização das respostas que são geradas.

A construção dos *prompts* permite, assim, controlar melhor a qualidade das respostas, reduzir ambiguidades e assegurar que a informação gerada responde de forma clara e objetiva às necessidades específicas do sistema.

Existem diversos tipos de técnicas de *prompt engineering* para ajudar a compreender melhor como comunicar de forma eficaz com *LLMs* e orientá-los de forma apropriada. Estes passam por *Zero-Shot Learning*, *One-Shot Learning*, *Few-Shot Learning* e *Chain-of-thought* [Sekiri, 2024, Aryani, 2023].

Zero-Shot Learning

Zero-Shot Learning refere-se à capacidade dos *LLMs* realizarem diversas tarefas novas sem que se forneçam certos tipos de exemplos ou treinos específicos para o resultado dessas tarefas. Esta técnica assenta apenas no conhecimento do modelo, sendo suficiente fornecer unicamente instruções objetivas.

Para que a técnica de *Zero-Shot Learning* funcione da forma correta, alguns fatores têm de ser considerados. Primeiramente, é fundamental que o *prompt* fornecido ao modelo seja claro, objetivo e suficientemente descritivo da tarefa pretendida, dependendo do contexto em que está a ser inserido. O facto de que nenhum exemplo concreto seja fornecido implica que o modelo deve compreender a tarefa apenas com base na descrição textual da instrução. A escolha de vocabulário e estrutura na criação do *prompt* irá influenciar de forma significativa a qualidade da resposta. Por fim, a avaliação dos resultados deve ser feita com critérios bem definidos, dado que a ausência de exemplos pode introduzir maior variabilidade nas respostas geradas.

Adicionalmente, é importante considerar que, apesar da sua flexibilidade, a técnica *Zero-Shot Learning* pode produzir respostas menos precisas ou contextualmente inadequadas em tarefas complexas. Nestes casos, torna-se recomendável redefinir o *prompt*, ou mesmo recorrer a técnicas complementares como o *Few-Shot Learning*, quando a tarefa exigir maior controlo sobre o formato ou o conteúdo da resposta.

One-Shot Learning

One-Shot Learning corresponde a uma abordagem em que o modelo recebe um único exemplo de entrada e saída antes de ser solicitado a realizar uma

tarefa. Este exemplo serve como referência para o modelo compreender o formato esperado e o contexto da tarefa, permitindo generalizar a partir desse único exemplo.

Few-Shot Learning

A ténica *Few-Shot Learning* consiste em fornecer ao modelo de linguagem alguns exemplos, normalmente entre dois a cinco, para averiguar a tarefa que se pretende que execute. Estes exemplos funcionam como guias, para permitir que o modelo consiga identificar padrões, estilo e estrutura das respostas esperadas.

Chain-of-thought

A técnica *Chain-of-Thought Prompting* incentiva o modelo a explicitar o seu raciocínio passo a passo antes de chegar a uma conclusão. Ao organizar o pensamento por etapas, o modelo consegue apresentar soluções mais fundamentadas e fiáveis.

2.1.6 Tecnologias Utilizadas

O *Docker* "fornece uma interface gráfica simples e intuitiva que permite gerir os seus contentores, aplicações e imagens diretamente a partir da própria máquina". [1] ([Docker, 2013], tradução própria).

É uma ferramenta que facilita o desenvolvimento, execução e gestão de aplicações através de um sistema de contentores. Estes contentores permitem isolar a aplicação, juntamente com todas as suas dependências, do sistema operativo subjacente, criando um ambiente de execução autónomo e replicável. Esta abordagem assegura que a aplicação se comporte de forma consistente em diferentes ambientes, eliminando problemas de incompatibilidade entre sistemas de desenvolvimento, teste e produção.

Para esse efeito, essa feramenta foi utilizada no projeto para garantir a execução isolada e consistente da aplicação *web* e dos agentes em qualquer ambiente.

[1] "It provides a straightforward GUI that lets you manage your containers, applications, and images directly from your machine."

2.1.7 Trabalho Relacionado

Apesar do desenvolvimento do projeto ser focado na área da criminologia, existem diversos outros projetos de investigação que recorrem a metodologias semelhantes, mas que partilham o mesmo objetivo fundamental: a automatização da análise e interpretação de documentos complexos através de modelos de linguagem e inteligência artificial, abaixo mencionados.

Applying generative AI with retrieval augmented generation to summarize and extract key clinical information from electronic health records

Esta pesquisa [Alkhalaf et al., 2024] explora a aplicação de modelos de linguagem, neste caso, o *Llama* 2 13B, para tarefas de resumo automático e extração da informação a partir de registos clínicos eletrónicos. A abordagem baseia-se em *zero-shot prompting* e é posteriormente comparada com *Retrieval-Augmented Generation* para melhorar a precisão e relevância das respostas.

Summarization of Large Text Volumes Using Large Language Models

Por outro lado, este artigo [Betrián e Kaltenbrunner, 2024] aborda a limitação de contexto dos *LLMs* através da proposta de três *pipelines* distintos. O primeiro, de natureza recursiva, divide o texto original em segmentos que são resumidos individualmente. O segundo *pipeline* é extrativo, recorrendo a técnicas automáticas para selecionar previamente os excertos mais relevantes. Por fim, o terceiro combina ambos num modelo híbrido. Neste artigo, o *Llama* 2, é utilizado para essa sumarização.

Chapter 3

Modelo Proposto

O modelo proposto neste projeto, resulta de uma progressão lógica que tem início na análise dos casos de utilização determinados e na análise dos requisitos. O objetivo desta secção do projeto é conseguir explicar de que forma que o sistema desenvolvido funciona, passando por todos os seus requisitos funcionais e não funcionais, apresentando também o respetivo modelo de casos de utilização proposto.

3.1 Requisitos

Os requisitos essenciais para este sistema dividem-se em dois grupos distintos, funcionais e não funcionais.

A identificação dos requisitos foi realizada após a análise das necessidades dos inspetores e dos objetivos gerais do projeto e os mesmos estão apresentados na Tabela 3.1 e 3.2

3.1.1 Requisitos Funcionais

No contexto do projeto, o foco principal é a gestão eficiente de processos-crime por parte dos inspetores, sendo essencial garantir a viabilidade e a fluidez das interações com o sistema.

A submissão de novos processos, como se pode averiguar na Tabela 3.1 é um dos requisitos fundamentais, visto que o sistema deve permitir que os inspetores insiram os novos processos-crime a resumir através da interface. Esta funcionalidade é a mais importante, dado que estabelece o ponto inicial para as interações posteriores.

Requisito	*Função*	*Categoria*
$RF1$	Submeter um novo processo	Visível
$RF2$	Gerar relatório	Visível
$RF3$	Permitir validação, edição e remoção	Visível
$RF4$	Armazenar processos	Invisível

Table 3.1: Tabela de Requisitos funcionais

Outro requisito essencial consiste na geração automática de relatórios. Após a submissão e o processamento dos processos-crime, o sistema deve ser capaz de apresentar a informação de modo estruturado e organizado via relatórios.

Para garantir que a informação gerada é factual, é necessário que o sistema permita a validação por meio da edição dos dados via inspetores. Esta funcionalidade garante que qualquer erro ou omissão de dados importantes possam ser corrigidos de modo a manter a integridade das informações.

Por fim, o sistema deve dispor da capacidade de armazenar os processos que lhe são inseridos, de modo a manter a preservação da informação inicial para, posteriormente, permitir comparações entre os ficheiros originais e o resultado gerado.

3.1.2 Requisitos Não Funcionais

Para além das funcionalidades principais, é essencial que o sistema respeite um conjunto de requisitos não funcionais que asseguram a sua qualidade e usabilidade. Estes requisitos complementam os comportamentos funcionais, garantindo que a utilização do sistema decorre de forma competente para os inspetores.

Requisito	*Função*	*Categoria*
$RnF1$	Apresentar formulários intuitivos	Visível
$RnF2$	Garantir compatibilidade com Sistemas Operativos	Invísivel
$RnF3$	Validar ficheiros submetidos	Invísivel

Table 3.2: Tabela de Requisitos funcionais

Um dos principais aspetos a considerar, como pode ser observado pela Tabela 3.2, é a usabilidade da interface, especialmente no momento de inserção de processos-crime. A aplicação deve apresentar caixas de diálogo e formulários claros, permitindo que os utilizadores compreendam facilmente cada passo, tendo em conta a formação profissional dos inspetores.

O sistema deve também garantir compatibilidade com diferentes sistemas operativos, nomeadamente Windows, Linux e macOS, permitindo o seu uso em contextos diversos sem necessidade de adaptações técnicas.

Por fim, a validação de ficheiros submetidos é outro requisito não funcional relevante. Apenas documentos em formatos suportados, neste caso *.docx* e *.pdf*, devem ser aceites, assegurando assim o processamento correto dos conteúdos por parte do sistema. Esta validação evita possíveis falhas que possam vir a surgir.

3.2 Casos de Utilização

Inicialmente, o desenvolvimento deste projeto foi orientado por um único objetivo: facilitar o trabalho dos inspetores na redação de relatórios criminais, através da automatização da tarefa de resumir documentos associados a processos-crime. Para isso, adotou-se um modelo simplificado de casos de utilização, centrado exclusivamente na funcionalidade de gerar o relatório criminal final a partir dos ficheiros fornecidos.

No entanto, à medida que o projeto evoluiu e foi sendo testado em cenários mais realistas, tornou-se evidente que esta abordagem única não seria suficiente para cobrir todas as necessidades práticas de um utilizador final. Com base no *feedback* obtido e na análise do fluxo de trabalho típico de um inspetor da Polícia Judiciária, foi necessário reformular o modelo inicial.

Assim, a aplicação *web* foi adaptada para incorporar estas novas funcionalidades pensadas, mais alinhadas com a realidade da criação de um relatório criminal. Estas funcionalidades adicionais foram integradas sob a forma de novos casos de utilização, tais como a visualização de ficheiros e respostas geradas, edição dessas mesmas respostas e do relatório final, e, por fim, a possibilidade de realizar *download* do relatório final gerado.

Desta forma, o modelo de casos de utilização evoluiu de uma estrutura simplificada para um modelo mais modular, ajustado às necessidades reais,

permitindo ao sistema oferecer um suporte mais completo a todos os profissionais envolvidos.

A Figura 3.1 demonstra todos os casos de utilização que foram desenvolvidos.

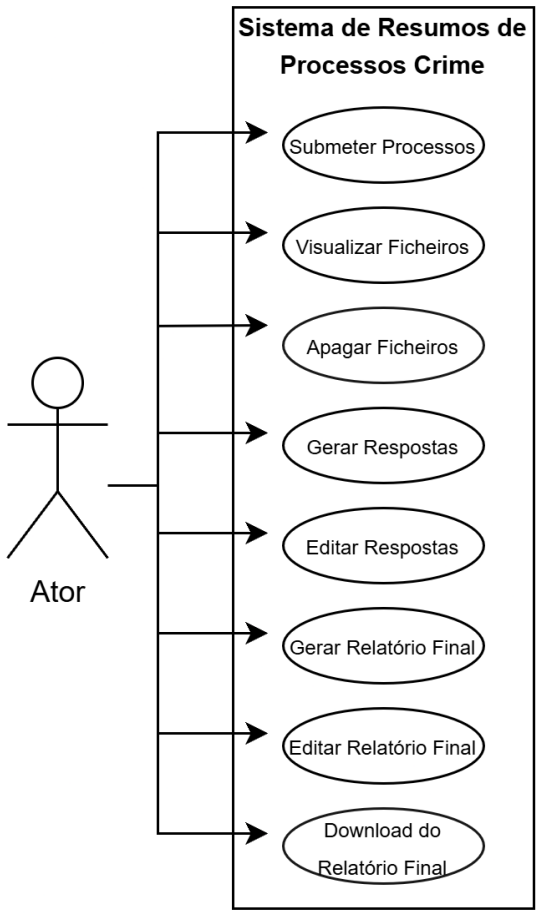


Figure 3.1: Modelo de Casos de Utilização

Como ilustrado na Figura 3.1, identificam-se oito tipos distintos de casos de utilização, cada um representando uma funcionalidade essencial no contexto da aplicação desenvolvida.

Submeter Processos

A submissão de processos permite ao inspetor carregar documentos relevantes diretamente na aplicação *web*. Esta funcionalidade é essencial para iniciar a análise dos ficheiros. Garante ainda que toda a informação é integrada de forma estruturada no sistema, e que todos os ficheiros inseridos coincidem com os que são permitidos. Este caso de utilização é iniciado com o clique no botão de submissão, permitindo ao inspetor selecionar os ficheiros desejados, desde que todos estejam associados ao mesmo processo-crime.

Visualizar Ficheiros

A visualização de ficheiros permite ao inspetor consultar os documentos previamente submetidos na aplicação. Esta funcionalidade é essencial para confirmar o conteúdo e para, posteriormente, acompanhar o progresso da análise automática. Este caso é realizado através de ícones representativos de cada documento, permitindo ao inspetor, com um simples clique, aceder a uma nova página onde pode consultar integralmente o conteúdo de cada ficheiro submetido.

Apagar Ficheiros

A funcionalidade de apagar ficheiros permite que o inspetor remova documentos submetidos anteriormente, a mesma é fundamental para garantir o controlo sobre os dados carregados, especialmente em situações de erro ou substituição de ficheiros. A ação é realizada de forma simples, através de um botão associado a cada ficheiro, permitindo ao utilizador gerir de forma eficaz o conteúdo disponível.

Gerar Respostas

A funcionalidade de gerar respostas permite ao inspetor obter automaticamente um resumo de cada um dos ficheiros submetidos, onde constam todos os dados mais relevantes do mesmo. Esta operação é iniciada com um clique no botão designado para esse fim, simplificando o processo de análise e acelerando a produção de documentação relevante.

Editar Respostas

A possibilidade de edição de respostas permite ao inspetor ajustar manualmente os conteúdos gerados automaticamente. Após a geração, cada resposta pode ser revista e editada diretamente na interface gráfica, garantindo a correção, clareza e adequação técnica a ser usufruída no relatório final. Para isso, ao conseguir visualizar a resposta gerada numa nova página, o inspetor tem a possibilidade de editar a respetiva resposta e guardar o seu conteúdo.

Gerar Relatório Final

A funcionalidade de se conseguir gerar um relatório final permite compilar automaticamente todas as respostas previamente produzidas e/ou editadas num único documento estruturado, este estruturado de forma cronológica. Esta ação é realizada através do clique no botão com esse efeito, que inicia a consolidação dos conteúdos numa versão final pronta para consulta, edição ou exportação.

Editar Relatório Final

A funcionalidade de editar o relatório final é semelhante à de editar respostas, permitindo ao inspetor ajustar manualmente o conteúdo do documento final consolidado. Esta ação garante que, mesmo após a geração automática, o relatório possa ser editado, corrigido ou complementado antes da sua validação final.

Download do Relatório Final

O caso de utilização de se poder realizar o *download* do relatório final permite ao inspetor obter uma cópia local do documento gerado, em formato *.docx*. Esta opção assegura que o trabalho produzido na aplicação possa ser facilmente exportado e utilizado fora do sistema.

3.3 Abordagem

O apêndice A contém a arquitetura geral do sistema. A sua referência noutras secções do relatório serve para ilustrar visualmente a estrutura e a relação

entre os componentes descritos, facilitando a compreensão do funcionamento interno do sistema.

Uma das principais inovações desenvolvidas neste projeto foi a criação de *modelfiles* específicos, concebidos para definir agentes capazes de interpretar e gerar automaticamente texto a partir de documentos legais. Estes ficheiros de configuração foram desenvolvidos com o objetivo de garantir resumos mais fiáveis, adaptados ao contexto jurídico e ajustados ao tipo de peça processual analisada.

No desenvolvimento do sistema, foram definidos dois *modelfiles* distintos: Resumo Geral e Resumo Cronológico, presentes nos apêndices B e C, respetivamente. Cada um destes ficheiros inclui instruções claras, via um *system prompt*, sobre a função esperada a realizar pelo modelo, a estrutura desejada na resposta e as restrições linguísticas e funcionais a aplicar. A sua parametrização técnica foi também cuidadosamente ajustada para evitar criatividade excessiva, repetições indesejadas e para garantir uma análise coerente e focada no conteúdo essencial.

O agente Resumo Geral B foi configurado para permitir ao modelo conseguir identificar, classificar e resumir documentos recebidos individualmente, tendo por base seis tipos de ficheiros comuns num processo-crime: Capa de Inquérito, Cota, Inquirição, Termo de Juntada, Notificação ao Lesado e Informação de Pesquisas em Fontes Abertas. O sistema instrui o modelo a adotar o papel de um inspetor da Polícia Judiciária, a interpretar corretamente o conteúdo textual e a gerar resumos objetivos, com linguagem profissional, clara e neutra, utilizando sempre um léxico do Português Europeu.

Por outro lado, o Resumo Cronológico C foi criado para ordenar a informação de forma sequencial. Este ficheiro instrui o modelo a reorganizar o conteúdo dos documentos com base numa estrutura fixa de um relatório criminal final que é composto por seis secções: Crime, Sujeitos Processuais, Descrição dos Factos, *Modus Operandi*, Diligências de Investigação e Conclusão.

Ambos os *modelfiles* foram configurados com parâmetros técnicos consistentes (*temperature*, *top_k*, *top_p*, *repeat_penalty*, entre outros), que garantem que os resultados gerados apresentem respostas satisfatórias.

No âmbito deste projeto, recorreu-se à técnica de *Zero-Shot Prompting*. Através de *prompts* cuidadosamente estruturados, foi possível orien-

tar o modelo na identificação, sumarização e reorganização de informação sensível. Esta metodologia permitiu avaliar a capacidade do modelo para generalizar e interpretar novos conteúdos com precisão, sem necessidade de treino adicional.

Chapter 4

Implementação do Modelo

O Capítulo 4 visa a implementação concreta do modelo proposto. Após o enquadramento teórico, mencionado no Capítulo 2, e a definição das componentes conceptuais e funcionais, mencionadas no Capítulo 3, este capítulo vai detalhar as decisões técnicas assumidas, aprofundar o porquê das bibliotecas utilizadas e toda a arquitetura adotada ao longo do desenvolvimento do projeto. São descritas todas as funcionalidades desenvolvidas no âmbito deste projeto, com o nível necessário para que o leitor compreenda com precisão o seu funcionamento e o respetivo suporte tecnológico.

4.1 Primeira Interação com o Flask

No início do desenvolvimento da aplicação *web*, a abordagem elaborada passou, naturalmente, pela mais simples e direta possível. Um primeiro contacto com a biblioteca *Flask*, mostrou a tendência de concentrar toda a lógica da aplicação num único ficheiro. Esta decisão refletia o grau inicial de familiaridade com a biblioteca e com os princípios básicos do desenvolvimento de aplicações *web* em *Python*.

Num primeiro momento, e como forma de introdução prática ao desenvolvimento com *Python* e à utilização do modelo **LLM** disponibilizado pelo *Ollama*, foi construído um protótipo inicial com o objetivo simples de compreender o funcionamento da *API*. Para isso, recorreu-se ao exemplo de código disponibilizado pelo próprio *Ollama* [Ollama, 2025], excerto de código mencionado na listagem 1, que permitia estabelecer comunicação básica com o serviço.

Essa implementação inicial foi deliberadamente simples e não estruturada, consistindo essencialmente num pequeno *script* que, após a configuração da chave de *API*, enviava uma prompt direta ao modelo e apresentava a resposta devolvida. Esta abordagem permitiu a exploração, de forma experimental, o comportamento do modelo face a diferentes tipos de instruções. Esta experiência foi fundamental para sedimentar os conhecimentos iniciais sobre integração com modelos de linguagem em *Python* e para perceber a importância de uma melhor estruturação dos pedidos, tanto a nível técnico como semântico. Este primeiro contacto informal foi, assim, o ponto de partida que conduziu, posteriormente, à necessidade de desenvolver uma aplicação *web* mais robusta.

Código 1: Exemplo de comunicação com o *Ollama*

```
1  import requests
2
3  url = "http://localhost:11434/api/generate"
4  payload = {
5      "model": "llama3",
6      "prompt": "Es um Inspetor da Policia Judiciaria e
           tens de falar em Portugues de Portugal",
7      "stream": False
8  }
9
10 response = requests.post(url, json=payload)
11 print(response.json()["response"])
```

4.2 Desenvolvimento da implementação

Apesar de este protótipo inicial ter permitido validar a comunicação com o modelo *LLM* e confirmar a capacidade de gerar texto coerente a partir de instruções simples, rapidamente identificaram-se algumas limitações práticas.

Em primeiro lugar, o processo era inteiramente manual, ou seja, o ficheiro a ser analisado tinha de ser aberto pelo utilizador, o texto copiado e inserido diretamente no *payload* da chamada da *API*. Esta abordagem, além de ineficiente, introduzia uma forte margem de erro, como a possibilidade de formatação incorreta, a perda de conteúdo ou a inserção de texto irrelevante. Adicionalmente, não havia qualquer persistência da informação — cada execução era isolada e descartável.

Para corrigir estas limitações, iniciou-se uma reformulação do sistema no sentido de permitir a submissão de ficheiros. Para isso, foi criado um formulário *HTML* básico que permitia ao utilizador selecionar um ou mais documentos do seu sistema de ficheiros e enviá-los diretamente para o servidor.

No lado do servidor *Flask*, foi desenvolvido um primeiro mecanismo de *upload* que armazenava temporariamente os ficheiros submetidos numa pasta local.

Com esta evolução, eliminou-se a necessidade de copiar manualmente o conteúdo dos documentos, reduzindo erros e tornando o processo mais fluído. Esta melhoria aumentou a fiabilidade do sistema ao permitir que os ficheiros fossem submetidos diretamente, sem intervenção manual na escrita dos dados, garantindo assim maior consistência e eficiência no tratamento da informação.

Outro problema identificado prendeu-se com a ausência total de interface gráfica. A interação ocorria unicamente através do terminal, o que se revelou inadequado para o público-alvo do sistema, neste caso, inspetores da Polícia Judiciária, que não devem depender de conhecimentos técnicos para utilizar a ferramenta. Este constrangimento levantou a necessidade urgente de desenvolver uma interface mais acessível, intuitiva e visualmente orientada para o fluxo de trabalho real dos utilizadores.

Para dar resposta a esta limitação, foi desenvolvido um primeiro protótipo de interface *web*. Esta interface permitia ao utilizador selecionar e submeter múltiplos ficheiros diretamente através do *browser*. No entanto, numa fase inicial, o comportamento da aplicação era ainda bastante simplificado, ou seja, assim que os ficheiros eram enviados, os modelos *LLM* eram automaticamente executados para gerar os resumos, sem qualquer intervenção intermédia do utilizador.

Esta abordagem, apesar de funcional, não permitia ao inspetor consultar previamente o conteúdo extraído antes da compilação do relatório final. Ainda assim, constituiu um avanço importante face à versão totalmente manual, pois eliminou a necessidade de interações por linha de comandos e ofereceu uma experiência de utilização mais próxima do ambiente real de trabalho.

Outra das limitações identificadas prendeu-se com a forma como os ficheiros

eram processados após o envio. Na versão inicial da interface web, os documentos submetidos eram automaticamente enviados para os *LLMs*, que geravam de imediato os resumos correspondentes a cada um dos ficheiros. Esta abordagem impedia que o inspetor tivesse qualquer controlo sobre o processo, não sendo possível visualizar previamente os ficheiros nem confirmar o seu conteúdo antes de gerar os resumos. Esta falta de transparência comprometia a confiança no sistema e dificultava a validação da informação a ser resumida.

Para ultrapassar esta limitação, foi implementada uma nova secção na interface, podendo ser observada pela Figura 4.2, que permite ao utilizador visualizar a lista de ficheiros submetidos, com a possibilidade de selecionar individualmente cada um e conseguir visualizar o seu conteúdo, o contexto apresentado na Figura 4.3, antes de prosseguir com a análise automática. Esta alteração introduziu uma camada essencial de validação e controlo, aproximando o sistema das reais necessidades operacionais dos inspetores.

Adicionalmente, foi também identificada uma limitação relacionada com a ausência de visibilidade sobre as respostas geradas para cada ficheiro submetido. No fluxo inicial da aplicação, os modelos geravam diretamente o relatório final a partir dos ficheiros, sem que o inspetor tivesse acesso às respostas individuais produzidas por cada resumo. Esta abordagem limitava o grau de confiança e transparência no processo, uma vez que não era possível verificar o que tinha sido efetivamente extraído de cada documento antes da compilação do relatório completo.

Para resolver esta falha, o sistema foi novamente reformulado de forma a incluir uma nova etapa intermediária. Após a submissão dos ficheiros, passou a ser apresentado um botão, apresentado na Figura 4.2, que permite, de forma explícita, gerar as respostas individuais para cada documento submetido. Só após esta ação, as respostas são exibidas numa nova secção da interface, permitindo ao utilizador visualizar, de forma clara e separada, o conteúdo que foi extraído. Além disso, apenas depois de todas as respostas estarem disponíveis é que surge o botão de geração do relatório final, garantindo assim que o utilizador tem primeiro a oportunidade de consultar os resumos parciais antes da produção do documento consolidado.

Apesar desta melhoria ter trazido maior transparência ao processo, persistia ainda uma limitação significativa, neste caso, a impossibilidade do in-

spetor ajustar ou corrigir as respostas individuais geradas pelos modelos antes da compilação do relatório final. Isto tornava o sistema demasiado rígido, uma vez que o utilizador não podia intervir nos conteúdos gerados, mesmo que detetasse erros, omissões ou interpretações menos adequadas à realidade do processo em questão.

Para responder a esta necessidade, foi introduzida a funcionalidade de edição das respostas individuais. Assim, para cada ficheiro submetido, após a geração da respetiva resposta pelo modelo, o sistema passou a disponibilizar uma área de texto editável, apresentada na Figura 4.5, associada a essa resposta. Esta funcionalidade permite ao inspetor rever, ajustar ou reescrever o conteúdo extraído, garantindo que o resumo final reflete com maior precisão as informações relevantes do caso. Esta etapa facultativa oferece maior flexibilidade ao utilizador, permitindo uma supervisão humana mais eficaz antes da geração do relatório criminal final.

Mesmo após permitir a edição das respostas individuais, identificou-se uma limitação adicional. O relatório final, gerado automaticamente a partir dessas respostas, não podia ser revisto nem ajustado pelo inspetor. O conteúdo era apresentado de forma definitiva, sem qualquer possibilidade de intervenção posterior, o que podia comprometer a clareza ou a correção do documento final.

Para resolver este problema, foi adicionada à aplicação a funcionalidade de edição do relatório final. Assim que o relatório cronológico é gerado, o sistema apresenta-o numa área de texto editável, apresentada na Figura 4.6, permitindo ao inspetor reler, corrigir, complementar ou reorganizar a informação antes de o considerar pronto.

Por fim, uma outra limitação importante dizia respeito à gestão dos ficheiros submetidos. Numa versão inicial do sistema, os documentos enviados eram automaticamente guardados numa pasta local específica. No entanto, esta abordagem não previa a possibilidade de iniciar um novo processo-crime sem reiniciar manualmente a aplicação *Flask*. Isto obrigava o inspetor a encerrar o sistema e voltar a executá-lo do zero sempre que pretendesse analisar um novo conjunto de documentos, o que se revelou pouco prático e contrário ao princípio de usabilidade pretendido.

Para ultrapassar esta limitação, foram implementadas duas soluções. Em primeiro lugar, o sistema passou a verificar, no momento da inicialização, se

existiam ficheiros antigos na pasta de uploads, procedendo automaticamente à sua remoção. Desta forma, garante-se que cada nova utilização começa com um ambiente limpo, sem necessidade de qualquer intervenção manual por parte do utilizador. Em segundo lugar, foi adicionada à interface a funcionalidade de remoção seletiva de ficheiros, como pode ser observado pela Figura 4.2. Caso o inspetor envie um ficheiro incorreto ou pretenda reorganizar o conjunto de documentos antes de gerar os resumos, pode agora eliminar individualmente qualquer ficheiro submetido, através de um botão específico. Esta funcionalidade proporciona maior controlo, flexibilidade e organização na preparação do material a ser analisado.

Embora não se tratasse de uma limitação identificada inicialmente, foi ainda adicionada à aplicação a possibilidade do inspetor descarregar o relatório final para a sua máquina. Esta funcionalidade permite guardar uma cópia local do documento gerado, facilitando a sua partilha ou integração com outros sistemas internos de gestão de processos. Esta contextualização do *download* de ficheiros pode ser observada na Figura 4.6.

4.3 Implementação da Solução Desenvolvida

Após a conclusão do protótipo inicial, focado na comunicação direta com o modelo *LLM* e na exploração funcional da *API*, avançou-se para a implementação da interface *HTML* que permitisse a interação efetiva entre o inspetor e o sistema. Esta interface tornou-se o ponto central da aplicação, possibilitando a submissão de ficheiros, a visualização dos resultados e, numa fase mais avançada, a edição, validação e remoção das respostas antes da geração do relatório final. Ao longo deste processo, foram também definidos e testados diversos exemplos de utilização, que demonstram o fluxo completo da submissão à geração do relatório.

4.3.1 Interface Web e Experiência do Utilizador

Uma primeira interação com a aplicação *web*, feita por um inspetor, é a submissão de ficheiros que pertençam ao mesmo processo-crime. Para isso, o inspetor apenas tem de clicar no botão "*Choose Files*", para determinar a quantia e quais os ficheiros a inserir e, de seguida, conseguir enviá-los via

o botão "Enviar Ficheiros", como pode ser averiguado na Figura 4.1. Os mesmos serão armazenados numa pasta *uploads*.



Figure 4.1: Página Inicial da Aplicação *Web*

De seguida, após a submissão dos ficheiros na aplicação *web*, o inspetor poderá visualizar todos os documentos carregados, desde que estejam num dos formatos suportados. Cada ficheiro é apresentado com um ícone representativo e, ao clicar sobre qualquer um deles, o inspetor poderá consultar o conteúdo extraído diretamente a partir do ficheiro submetido. Este contexto pode ser verificado, abaixo, na Figura 4.2.



Figure 4.2: Ficheiros Submetidos pelo Inspetor

Ao clicar em qualquer um dos ícones, o inspetor terá a possibilidade de ver o conteúdo extraído, de forma formatada, de cada um dos ficheiros submetidos e, se existir algum tipo de erro no envio de ficheiros, como por exemplo se tiver enviado um ficheiro que não pertença ao mesmo processo-crime de todo, o inspetor pode, ao carregar no botão de "Apagar", eliminar o ficheiro que anteriormente tinha sido armazenado. Para esse efeito, a Figura 4.3 é apresentada.

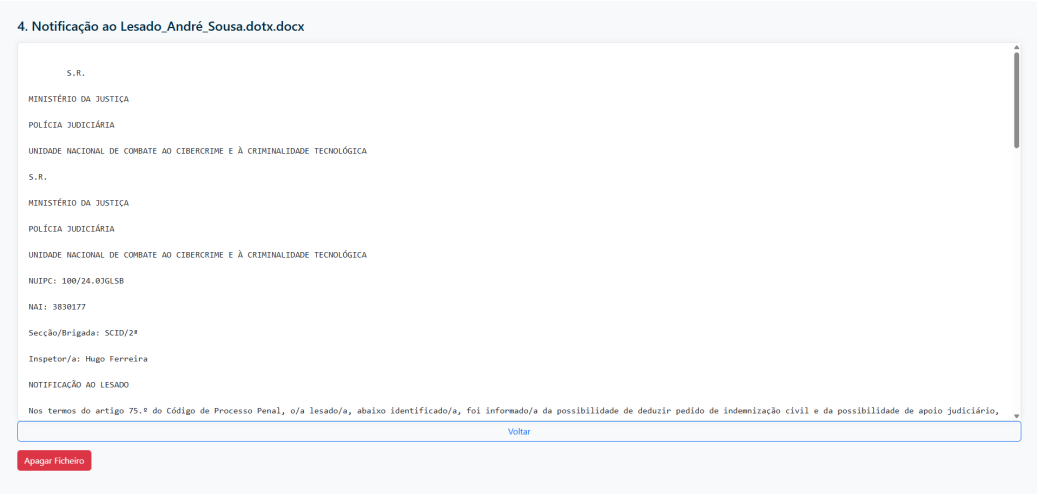


Figure 4.3: Visualização do Ficheiro submetido

Após a submissão e visualização dos ficheiros por parte do inspetor, a aplicação disponibiliza um botão que permite gerar automaticamente respostas/resumos com base no conteúdo carregado. Ao clicar nesse botão, é realizada uma comunicação com a *API* do *Ollama*, através de pedidos *HTTP* implementados em *Python*. Para tal, foi desenvolvido um agente *LLM* personalizado 3.3 que interage diretamente com o modelo alojado no *Ollama*.

Este agente é responsável por analisar o conteúdo de cada ficheiro e produzir resumos de caráter geral, adaptados à natureza da informação extraída. Os resultados são processados em segundo plano e, após um breve intervalo de tempo, apresentados na interface *web*, permitindo ao inspetor aceder de forma rápida e estruturada à síntese da documentação submetida.

Esta verificação é possível ser observada quando mencionada a Figura 4.4.



Figure 4.4: Respostas Geradas pelo agente criado

A visualização e edição da resposta gerada pode ser realizada pelo inspetor ao clicar em qualquer um dos ícones de respostas associadas a cada tipo de ficheiro inserido. É possível aceder a essa funcionalidade, como pode ser visto na Figura 4.5.



Figure 4.5: Possível visualização e edição das respostas geradas

Por fim, após a obtenção das respostas individuais para cada ficheiro submetido, e após a eventual edição manual por parte do inspetor, a aplicação disponibiliza a funcionalidade de um botão para "Gerar Relatório Final". Ao

clicar no mesmo, é realizada uma nova operação com base nos conteúdos já existentes, em que os resumos previamente validados são agregados de forma cronológica por um agente 3.3 também dedicado, concebido especificamente para essa tarefa.

O inspetor será redirecionado para uma nova página da aplicação onde poderá visualizar o conteúdo do relatório cronológico que foi gerado automaticamente. Esta página oferece, ainda, a possibilidade de editar diretamente o relatório final, permitindo ao utilizador aplicar correções, detalhes ou adicionar observações específicas antes de o considerar definitivo. Se assim quiser, também terá a possibilidade de realizar o download de uma cópia para a sua máquina de forma local.

Toda esta contextualização pode ser observada na Figura 4.6.



Figure 4.6: Possível visualização, edição e *download* do Relatório Final

4.3.2 Estrutura Modular e Lógica de Backend

Como mencionado na secção 3.3, no Capítulo 3, foi implementada uma estrutura *UML*, com o objetivo de planear os módulos, mais tarde implementados no *backend* do nosso projeto.

Tecnologias / Bibliotecas Utilizadas

Durante a fase de planeamento do projeto, foi proposta a utilização da linguagem de programação *Python* para o seu desenvolvimento. Esta escolha

baseou-se não só na escolha do orientador do projeto, mas, como também, em vários critérios, entre os quais se destaca a familiaridade do grupo de projeto com a linguagem, adquirida ao longo do percurso académico.

A escolha desta linguagem também mostrou a sua importância no que toca à sua compatibilidade com a biblioteca *Flask*, que, no desenvolvimento deste projeto, desempenhou um papel fundamental na criação de uma aplicação *web*. A sua simplicidade permitiu estruturar um serviço *web* funcional, facilitando a integração entre a interface e a lógica de tratamento de documentos.

A biblioteca *Flask* [Flask, 2025] "foi concebida para facilitar o início do desenvolvimento de forma rápida e simples, com capacidade para escalar até aplicações complexas" [1] ([Flask2.0.1, 2021], tradução própria). Esta biblioteca permite gerir rotas e lidar com pedidos e respostas, funcionando como base de comunicação entre o cliente, *frontend*, e o servidor, *backend*. Através do seu sistema de templates, foi possível gerir páginas *HTML* dinâmicas.

A comunicação que é feita com os agentes *LLM* é efetuada através da *API* [Ollama, 2025] disponibilizada pela ferramenta *Ollama*, que é executada localmente. Esta interação é executada via protocolo *HTTP*, ao utilizar um método *POST* para enviar os dados necessários a serem processados. O *Ollama* vai, então, processar esse pedido e devolver uma resposta em formato *JSON*, contendo a mesma extraída em texto. Este processo visa a automatização da análise de documentos de uma forma mais simplificada, local e eficiente, sem o recurso a serviços externos.

Para além da biblioteca *Flask*, a mesma responsável pela criação da aplicação *web*, foram também utilizadas várias outras bibliotecas com funcionalidades específicas para cada tipo de contexto no processamento e tratamento da informação. Entre elas destacam-se a *re*, para a limpeza e manipulação de texto, a *requests*, para a comunicação com *APIs* externas, a *os*, para a operação no sistema de ficheiros, as *docx2txt* e *fitz*, para a extração de conteúdo de documentos em formato *.docx* e *.pdf*, respetivamente, e por fim, a *docx*, para a criação de ficheiros *Word*.

[1] "It is designed to make getting started quick and easy, with the ability to scale up to complex applications"

Módulo Interface *Web - interface*

Este módulo tem como intuito disponibilizar a interface gráfica da aplicação, permitindo ao inspetor interagir de forma intuitiva e eficiente com o sistema. Através de páginas *web* construídas com *HTML* e *CSS*, o utilizador pode submeter ficheiros, visualizar conteúdos extraídos, aceder aos resumos gerados pelos agentes *LLM*, editar esses mesmos resumos, e por fim, consultar e editar o relatório final cronológico. Trata-se, portanto, da camada de apresentação da aplicação, que assegura a comunicação entre o utilizador e os restantes módulos funcionais do sistema.

Módulo Núcleo da Aplicação - *final report app*

O módulo *final_report_app* constitui o núcleo lógico da aplicação, sendo responsável por coordenar as funcionalidades principais do sistema. Este módulo agrega as diferentes componentes que tratam da leitura, validação, processamento e organização dos ficheiros submetidos, bem como da comunicação com os agentes *LLM* e da geração dos resumos e relatórios. Centraliza, assim, a lógica de negócio da aplicação, garantindo que todas as operações são realizadas de forma estruturada, modular e escalável.

Existem diferentes classes que necessitam a sua interpretação e estudo para determinar qual o objetivo em concreto de cada uma. Mais abaixo, o contexto de cada uma dessas classes, para o desenvolvimento da arquitetura do sistema, é apresentado.

1. ***final_report*** : Este módulo é responsável por inicializar toda a aplicação *web* desenvolvida em *Flask*. A função *create_app()* tem como objetivo configurar o ambiente da aplicação, começando pela definição dos diretorias de *templates* e ficheiros *static*, essenciais para o correto funcionamento da interface gráfica. Logo de seguida, é feita a limpeza da pasta de *uploads* através da classe *DeleteUploads*, mencionada no ponto 9, garantindo que, sempre que a aplicação é reiniciada, os ficheiros antigos não interferem com novas submissões. São também inicializadas as instâncias das classes *Manager*, ponto 2 e *ManagerLLM*, ponto 11, responsáveis, respetivamente, pela gestão de ficheiros submetidos e pela comunicação com os modelos de linguagem.

2. ***manager*** : Este módulo define a classe que centraliza toda a gestão de ficheiros submetidos na aplicação. Quando instanciada, esta classe prepara a pasta de *uploads* e garante a sua existência, instanciando também as componentes responsáveis pela validação de extensões a partir da classe *FileValidator*, mencionado no ponto 5, leitura e conversão de conteúdos, *DocParser*, no ponto 4, e limpeza de texto, *CleanText*, referida no ponto 10. O método *show_valid_files()* percorre a pasta de *uploads* e devolve apenas os ficheiros que têm extensões permitidas. Já o método *read_content()* permite, dado o nome de um ficheiro, extrair o seu conteúdo textual limpo e pronto a ser apresentado ao utilizador.

3. ***routes*** : Este módulo é responsável por gerir todas as rotas da interface da aplicação *web*. Irá lidar com a submissão de ficheiros por parte do inspetor, processando e validando os documentos carregados. Após o *upload*, os conteúdos dos ficheiros são extraídos, limpos e apresentados na interface. Com a funcionalidade de geração automática de resumos, o módulo comunica com agentes, obtendo sumários para cada ficheiro individual. Estes resumos podem ser visualizados e editados, sendo depois usados na criação de um relatório final estruturado cronologicamente, que o utilizador também pode editar e exportar em formato *.docx*. Além disso, permite eliminar ficheiros carregados. Este módulo representa, portanto, o ponto de ligação entre o *frontend* e a lógica de processamento dos ficheiros e resumos.

4. ***doc_parser*** : O módulo *DocParser* é responsável pela extração de texto a partir de documentos submetidos na aplicação, permitindo converter ficheiros *.docx*, *.doc* e *.pdf* para texto plano. Para tal, recorre às bibliotecas *docx2txt*, para ficheiros *Word* e *fitz*, para *PDFs*. Esta classe disponibiliza o método *convert_file2content()*, que identifica a extensão do ficheiro e encaminha o seu processamento para o método apropriado, *_read_doc()*, no caso de documentos *Word*, e *_read_pdf()*, para documentos *PDF*.

5. ***file_validator*** : O módulo *FileValidator* é responsável por assegurar que apenas ficheiros com extensões suportadas são aceites pela aplicação. A classe define, no seu construtor, um conjunto de extensões válidas, neste caso, *.docx*, *.doc* e *.pdf*. O método principal, *is_valid()*,

recebe um nome de ficheiro e verifica se este termina com uma das extensões permitidas.

6. ***llm*** : O módulo define a classe abstrata *LLM*, a qual estabelece uma estrutura base para qualquer implementação de um agente de linguagem natural que interaja com o servidor *Ollama*. Define os elementos essenciais de comunicação, nomeadamente o endereço da *API*, o *OLLAMA_URL*, e os cabeçalhos *HTTP* necessários para os pedidos *JSON*. Além disso, declara o método abstrato *process_file()*, que obriga qualquer subclasse a implementar a sua própria lógica de processamento de ficheiros.

7. ***cop_llm*** : Este módulo representa um agente *LLM* específico para o processamento individual de ficheiros, neste caso, o mencionado no ponto 3.3. Esta classe herda da interface abstrata *LLM* e concretiza o método *process_file()*, que permite enviar o conteúdo textual de um ficheiro para o modelo, alojado no servidor *Ollama*. A comunicação é realizada através de um pedido *HTTP POST*, em que o conteúdo do ficheiro é utilizado como *prompt*.

8. ***cop_chronologic*** : Este módulo implementa uma função principal sendo ela a geração de um resumo cronológico final utilizando o agente criado mencionado no ponto 3.3. Esta classe recebe como entrada um dicionário contendo os resumos individuais previamente gerados pelo agente *CopLLM*, ponto 7, e constrói um único *prompt* consolidado, organizando os resumos por nome de ficheiro. O conteúdo é então enviado, via pedido *HTTP POST*, para o modelo disponibilizado através da *API* do servidor *Ollama*. O resultado da resposta, após ser convertido de *JSON*, corresponde ao resumo final com os eventos ou informações ordenadas cronologicamente. Esta etapa é essencial para a criação do relatório final apresentado na aplicação.

9. ***delete_uploads*** : O módulo *DeleteUploads* tem como objetivo garantir uma inicialização limpa da aplicação sempre que o servidor é reiniciado. Para isso, implementa a funcionalidade de eliminação automática dos ficheiros previamente armazenados na pasta de *uploads*. Através da biblioteca *os*, o método *limpar_uploads()* consegue verificar se a pasta específica existe, percorre todos os ficheiros contidos na mesma e elim-

ina todos os que forem identificados como ficheiros individuais. Esta limpeza previne a acumulação de dados e assegura que a aplicação começa sempre num estado consistente e funcional.

10. ***clean_text*** : O módulo *CleanText* tem como objetivo limpar e formatar o texto extraído dos ficheiros, espaços desnecessários e normalizar a separação entre parágrafos. Isto garante uma apresentação mais clara e organizada do conteúdo ao inspetor.

11. ***manager_llm*** : O módulo *ManagerLLM* centraliza a gestão dos agentes *LLM* utilizados na aplicação. Ele instancia os modelos *CopLLM*, ponto 7, para gerar resumos individuais dos ficheiros e *CopChronologic*, ponto 8, para compilar esses resumos num único relatório final ordenando-o cronologicamente.

Depois de toda a arquitetura da aplicação estar implementada, optou-se por recorrer à ferramenta *Docker* como forma de empacotar o projeto. Esta decisão permitiu uma maior portabilidade e facilidade na distribuição da aplicação, evitando problemas de compatibilidade entre diferentes sistemas operativos ou ambientes de desenvolvimento. Com todos os módulos e dependências encapsulados num único ambiente isolado, tornou-se possível executar a aplicação com apenas alguns comandos, mesmo em máquinas externas onde o ambiente de desenvolvimento original não estivesse presente.

Chapter 5

Validação e Testes

Esta secção descreve a fase de validação e testes do sistema desenvolvido, com o objetivo de avaliar a sua eficiência e precisão no contexto de utilização previsto. A validação incide sobre as componentes apresentadas no Capítulo 3, verificando se os requisitos funcionais e não funcionais definidos foram cumpridos.

Foram realizados testes com diferentes tipos de ficheiros processuais, avaliando-se a capacidade do modelo em identificar corretamente o tipo de documento, resumir a informação de forma clara e organizar os conteúdos conforme as orientações definidas nos *modelfiles*.

O principal foco desta fase é garantir que o sistema responda com coerência e qualidade aos objetivos propostos, sendo capaz de apoiar o trabalho dos inspetores de forma eficaz e consistente.

5.1 Geração de Relatórios Finais

Nesta fase, foi testada a geração automática dos relatórios finais a partir dos resumos produzidos pelos agentes. O objetivo é verificar se os modelos seguiram corretamente as instruções definidas nos *modelfiles*, tanto no que diz respeito à estrutura como ao conteúdo. Encontram-se, então, nas listagens 2 e 3, os resultados finais de exemplos da geração de resumos de processos-crime.

Código 2: Exemplo de Relatório Gerado 1

```
1. Crime - Explica em termos simples o crime

```

```
3  O crime que esta sendo investigado e a Falsidade
       Informatica. Isso significa que alguem esta tentando
       enganar ou manipular informacoes eletronicas para
       obter vantagens indevidas.
4
5  A falsidade informatica pode envolver uma variedade de
       atividades, como:
6
7  Spoofing: enviar mensagens ou chamadas telefonicas
       fraudulentas com o objetivo de enganar as pessoas.
8
9  Phishing: enviar e-mails ou mensagens que parecem
       legitimas mas sao na verdade enviados por criminosos
       para roubar informacoes confidenciais.
10
11 Manipulacao de dados: alterar ou remover dados em
       sistemas computacionais para obter vantagens
       indevidas.
12
13 2. Sujeitos processuais - Identifica os intervenientes
       no processo e a sua funcao
14
15 Os sujeitos processuais neste caso sao:
16
17 Denunciante: Joaquim Trindade Afonso, um homem casado
       com filiacao a familia Manuel Albertino Afonso e
       Maria das Palmas Trindade.
18
19 Arguido: "VenderValor", um suspeito que esta sendo
       investigado pela falsidade informatica.
20
21 Testemunha: Andre Sousa, um tecnico de IT que foi
       notificado pelo denunciante sobre as chamadas
       telefonicas fraudulentas.
22
23 Inspetor/a: Hugo Ferreira, o inspetor associado ao
       inquerito que esta liderando a investigacao.
24
25 3. Descricao dos Factos - Resume os factos reportados,
       destacando os principais eventos e elementos
       temporais
26
27 Os fatos reportados neste caso incluem:
```

```
28
29 Em 01/02/2024, Joaquim Trindade Afonso comecou a receber
       chamadas telefonicas fraudulentas com o numero
   "934000111".
30
31 No dia seguinte, Andre Sousa foi notificado pelo
   denunciante sobre as chamadas telefonicas.
32
33 Em 14/02/2024, a Policia Judiciaria realizou uma
   inquiricao com Andre Sousa para coletar mais
   informacoes sobre o caso.
34
35 A investigacao tambem envolveu pesquisas em fontes
   abertas, incluindo a busca por informacoes online e a
       analise de dados.
36
37 4. Modus Operandi - Explica o modo como o crime foi
   alegadamente cometido
38
39 O modus operandi (modo de operar) do criminoso e
   desconhecido neste momento, mas parece que ele
   utilizou um numero de telefone falso ("934000111")
   para enviar chamadas telefonicas fraudulentas ao
   denunciante Joaquim Trindade Afonso.
40
41 Alem disso, ha indicios de que o criminoso pode ter
   utilizado tecnicas de phishing ou spoofing para
   enganar as pessoas.
42
43 5. Diligencias de Investigacao - Descreve as acoes
   tomadas pela policia ou autoridades, e os resultados
   obtidos
44
45 As acoes tomadas pela Policia Judiciaria incluem:
46
47 Notificacao do denunciante Joaquim Trindade Afonso sobre
       as chamadas telefonicas fraudulentas.
48
49 Realizacao de uma inquiricao com Andre Sousa para
   coletar mais informacoes sobre o caso.
50
51 Pesquisas em fontes abertas, incluindo a busca por
   informacoes online e a analise de dados.
```

```
52
53 Os resultados obtidos ate agora sao:
54
55 A identificacao do numero de telefone falso
      ("934000111") como o responsavel pelas chamadas
      telefonicas fraudulentas.
56
57 A notificacao da empresa "VenderValor" como um suspeito
      no caso.
58
59 6. Conclusao - Resume o desfecho da investigacao,
      referindo se ha materia suficiente para continuar ou
      arquivar, e o porque
60
61 A conclusao desta investigacao ainda esta pendente, mas
      os resultados obtidos indicam que ha uma base solida
      para prosseguir com a investigacao.
62
63 O denunciante Joaquim Trindade Afonso apresentou
      testemunhos convincentes sobre as chamadas
      telefonicas fraudulentas, e a identificacao do numero
       de telefone falso ("934000111") e um passo
      importante na investigacao.
64
65 No entanto, e necessario continuar a investigacao para
      coletar mais informacoes e provas concretas. A
      continuidade da investigacao e fundamental para
      esclarecer os fatos envolvidos no crime e alcancar um
       resultado justo.
```

Código 3: Exemplo de Relatório Gerado 2

```
1 0. Capa Inquerito.dotx.docx
2 Analise do Inquerito N 100/24.0JGLSB
3
4 O inquerito apresentado contem informacoes essenciais
      sobre o crime de falsidade informatica, que sera
      analisada para identificar os pontos-chave.
5
6 Capitulo I: Contexto e Objetivos
7
8 O inquerito foi aberto em data 01/02/2024 e encerrou-se
      em 14/02/2024.
9
```

```
A seccao responsavel pelo caso e a SCIDBRIGADA 2.

O inspetor associado ao caso e Hugo Ferreira (NUIPC:
   3830177).

1. Cota_Andre_Sousa_notificacao.docx
S.R.,

Analise dos ficheiros recebidos:

Capa de Inquerito: O NUIPC (Numero Unico de
   Identificacao do Processo-Crime) e 100/24.0JGLSB,
   indicando que este inquerito foi aberto em 2024 sob a
    sigla JGLSB, pertencente a Seccao SCID e Brigada 2
   da Policia Judiciaria.

Cota: Esta cota registra o acto de notificar Andre Sousa
    atraves do telefone 938765432 para ser inquirido no
   presente inquerito criminal. A pessoa mencionada
   expressou disponibilidade para ser ouvida na data de
   19/03/2024 as 15h00, permitindo tambem a
   possibilidade de acompanhamento por um advogado,
   conforme solicitado.

2. Inquiricao_Andre_Sousa.docx
S.R.,

Apos analisar o ficheiro "AUTO DE INQUIRICAO",
   identificamos os seguintes pontos-chave:

Capa: O documento apresenta um numero unico de
   identificacao do processo-crime (NUIPC) de 100/24.0
   JGLSB e uma seccao/brigada SCID/2.

Cota: A anotacao oficial no processo e a seguinte: "
   Procedeu-se a notificacao do arguido".

Inquiricao: Esta secao contem informacoes sobre a
   testemunha Andre Sousa, que foi ouvida pela Policia
   Judiciaria. As informacoes incluem:

Nome completo: Andre Sousa

```

```
Alcunha: Nao disponivel

Filiacao: Manuel Albertino Afonso (Pai); Maria das
   Palmas Trindade (Mae)

Nacionalidade: Portuguesa

Data de nascimento: 12/04/1987

Estado civil: Casado

Profissao: Tecnico de IT thfgjhyrjfjhf

A testemunha relata ter recebido chamadas telefonicas a
   partir de numeros desconhecidos, onde alguem afirmava
    que o numero do depoente era registado nas chamadas
   nao atendidas. Alega estar vitima de spoofing.

3. Termo de juntada docs..docx
S.R.,

A partir do Termo de Juntada apresentado, e possivel
   identificar os seguintes pontos-chave:

O NUIPC (Numero Unico de Identificacao do Processo-Crime
   ) e 100/24.0JGLSB.

A Seccao e a Brigada associadas sao SCID/2.

O Inspetor responsavel pelo caso e Hugo Ferreira.

O auto de inquiricao mencionado no Termo de Juntada
   parece ter sido anterior a troca de e-mails e as
   chamadas telefonicas que foram adicionados aos autos.
    As informacoes incluidas neste termo permitem uma
   analise mais detalhada dos passos legais adotados
   pela Policia Judiciaria durante o processo criminal
   em questao, destacando especialmente as medidas
   tomadas para garantir a integridade das provas
   coletadas.

Este Termo de Juntada conclui com um registro oficial da
    inclusao de novos documentos nos autos do caso,
```

```
   indicando que os elementos apresentados sao parte
   integrante do processo judicial.
64
65 4. Notificacao ao Lesado_Andre_Sousa.dotx.docx
66 S.R.,
67
68 A analise dos ficheiros enviados revela que o NUIPC (
   Numero Unico de Identificacao Processo-Crime) e
   100/24.0JGLSB, indicando um caso criminal com
   identificador unico e data de inicio em abril de
   2024. Alem disso, a Seccao/Brigada associada e a SCID
   /2.
69
70 O documento notifica ao lesado, Andre Sousa, sobre a
   possibilidade de deduzir pedido de indemnizacao civil
    e apoio judiciario no processo penal. A Lei n.
   34/2004, de 29 de julho, preve as regras para o
   procedimento do pedido de indemnizacao civil.
71
72 Os artigos relevantes mencionados neste ficheiro sao:
73
74 Artigo 75.: Dever de informacao - As autoridades
   judiciarias devem informar os lesados sobre a
   possibilidade de deduzirem pedido de indemnizacao
   civil.
75
76 Artigo 76.: Representacao - O lesado pode fazer-se
   representar por advogado ou nao e obrigatorio em
   todos os casos.
77
78 Artigo 77.: Formulacao do pedido - A forma como se
   apresenta o pedido de indemnizacao civil, dependendo
   da situacao especifica do caso e do valor do pedido.
79
80 O documento conclui com uma nota de que o lesado Andre
   Sousa foi notificado dos seus direitos e deveres no
   processo penal, e podera requerer apoio judiciario
   nos servicos de atendimento publico da Seguranca
   Social.
81
82 Assinatura,
83 Hugo Ferreira
84 Inspetor/a
```

```
85
86  5. Informacao_pesquisas em fontes abertas_333.docx
87  S.R.
88
89  MINISTERIO DA JUSTICA
90  POLICIA JUDICIARIA
91  UNIDADE NACIONAL DE COMBATE AO CIBERCRIME E A
       CRIMINALIDADE TECNOLOGICA
92
93  ANALISE DO FICHEIRO "INFORMACAO PESQUISAS EM FONTES
       ABERTAS"
94
95  O ficheiro apresentado contem uma informacao oficial
       elaborada por um agente da Policia Judiciaria, que
       realizou pesquisa em fontes abertas no numero de
       telefone 934000111. A analise do conteudo revela os
       seguintes pontos-chave:
96
97  O numero de telefone foi associado a empresa VenderValor
       , como contacto telefonico;
98
99  A pesquisa foi feita atraves do site www.google.com.
100
101 A data da pesquisa e o dia 02/04/2024.
102
103 Em resumo, este ficheiro fornece informacoes sobre a
       associacao entre um numero de telefone e uma empresa,
        bem como as medidas tomadas pela Policia Judiciaria
       para investigar esta informacao. Esta analise
       demonstra que o objetivo da unidade nacional de
       combate ao cibercrime e a criminalidade tecnologica e
        garantir a seguranca publica e proteger os direitos
       das vitimas.
104
105 A importancia destas informacoes esta no contexto do
       inquerito em curso, onde se busca entender melhor as
       relacoes entre individuos e empresas, especialmente
       aquelas que podem estar envolvidas na actividade
       ilicita.
106
107 6. Conclusao
108 O inquerito N 100 24.0JGLSB apresentado contem uma serie
        de documentos detalhados sobre um caso suspeito de
```

```
   falsificacao online. O documento principal aborda o
   crime de falsidade informatica, destacando-se a
   importancia da cooperacao internacional para combater
    este fenomeno global. A analise dos ficheiros revela
    pontos-chave como a identificacao do numero de
   telefone associado a empresa VenderValor, bem como as
    medidas tomadas pela Policia Judiciaria para
   investigar esta informacao e garantir a seguranca
   publica.
109
110 A conclusao destes documentos e que a luta contra a
   falsificacao online requer esforcos conjuntos entre
   os paises membros das organizacoes internacionais
   envolvidas na sua prevencao.
```

5.2 Comparação com um Relatório Real

No âmbito da validação do sistema, foi desenvolvido um *modelfile* específico com o objetivo de comparar dois tipos distintos de relatórios finais de processos criminais: um redigido por um inspetor humano e outro gerado automaticamente através de ambos os agentes criados anteriormente, mencionado no ponto 3.3.

A criação deste *modelfile* permitiu não só testar a capacidade de análise comparativa do sistema, como também validar a consistência e qualidade dos relatórios gerados. Foram tidas em conta diferenças ao nível da organização da informação, clareza da linguagem, uso de expressões jurídicas, e coerência temporal dos factos apresentados. O objetivo principal foi garantir que o sistema consiga identificar padrões característicos da escrita humana em contraste com a geração automática.

A comparação, feita pelo *modelfile*, forneceu um resultado verídico, pois o mesmo conseguiu distinguir qual seria o criado pelo inspetor e qual o gerado de forma automática. O resultado é apresentado na Figura 5.1.

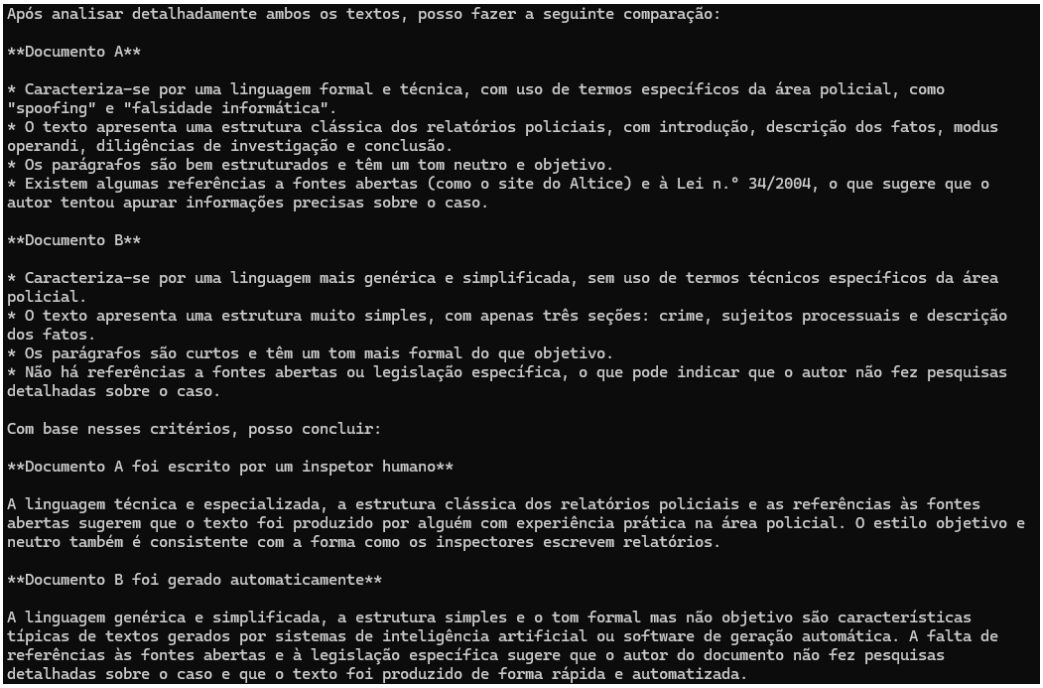


Figure 5.1: Comparação entre Relatórios

Apesar da comparação não ter produzido resultados satisfatórios neste caso concreto, ou seja, o sistema conseguiu identificar corretamente qual dos relatórios foi escrito por um humano e qual foi gerado automaticamente, esta abordagem abre caminho para trabalhos futuros. Com o avanço de modelos de linguagem de maior dimensão e com mais parâmetros, será possível explorar esta comparação de forma mais rigorosa.

Chapter 6

Conclusões e Trabalho Futuro

6.1 Conclusões

O desenvolvimento deste projeto refletiu uma abordagem inovadora à utilização de inteligência artificial no apoio à análise de processos-crime, explorando as potencialidades dos *LLMs*, concretamente através da técnica de *Zero-Shot Prompting*. Apesar da proposta inicial contemplar a implementação de um sistema *RAG* (*Retrieval-Augmented Generation*), optou-se por uma simplificação estratégica do modelo, focando o desenvolvimento numa solução mais leve e direta, que pudesse operar exclusivamente com base no conhecimento pessoal dos *LLMs*, sem necessidade de uma base de documentos.

Ao longo do projeto, foi desenvolvido um sistema capaz de interpretar automaticamente conteúdos processuais provenientes de diferentes tipologias documentais como cotas, inquirições, termos de juntada, notificações e outros documentos jurídicos, permitindo a sua análise automatizada, resumo objetivo e reorganização cronológica. Esta funcionalidade revelou-se especialmente relevante no contexto judiciário, onde a leitura extensiva e análise cuidadosa de processos pode ser lenta e suscetível a falhas humanas.

Através da utilização da biblioteca *Flask*, foi possível implementar uma aplicação *web* funcional e modular, capaz de gerir *uploads* de ficheiros, invocar *LLMs* localmente através da ferramenta *Ollama*, e apresentar os resultados ao utilizador. O sistema foi ainda projetado para ser facilmente integrável em ambientes institucionais, respeitando princípios de modularidade, portabilidade e segurança, nomeadamente através do uso de contentores *Docker*.

A substituição do mecanismo *RAG* revelou-se acertada, dado que permitiu atingir os objetivos funcionais essenciais do projeto com menos complexidade técnica, ao mesmo tempo que garantiu uma boa qualidade nos resultados gerados. A utilização de *prompts* cuidadosamente estruturados, em conjunto com parâmetros ajustados nos agentes *LLM*, permitiu obter resumos coesos, corretos do ponto de vista linguístico e respeitadores das convenções da linguagem jurídica portuguesa.

Durante o processo de desenvolvimento, foram tomadas decisões tecnológicas fundamentadas. O uso de bibliotecas como *requests*, *re*, *os*, *docx2txt* e *fitz* permitiu implementar funcionalidades como limpeza textual e extração de conteúdos de documentos *PDF* e *Word*. Estes elementos contribuíram para garantir a eficiência do sistema.

Em termos académicos e práticos, este projeto constituiu uma oportunidade singular de aplicar conhecimentos nas áreas de engenharia informática, inteligência artificial, engenharia de software e ciência de dados, com impacto direto num domínio sensível como o da justiça criminal. A colaboração com a Polícia Judiciária reforçou ainda mais o carácter aplicado e a relevância social desta proposta.

Em suma, o sistema desenvolvido demonstra que, mesmo sem recorrer a mecanismos complexos de recuperação de contexto como o *RAG*, é possível aplicar *LLMs* em ambientes reais, garantindo ganhos significativos de tempo, qualidade e rigor na produção de relatórios criminais finais. Este projeto abre portas para futuras extensões como integração com mecanismos de verificação factual, apoio a tradução jurídica automática ou sumarização por tipo de peça processual, que poderão continuar a inovar a forma como a tecnologia serve a justiça.

6.2 Trabalho Futuro

Este projeto estará disponível para todos os inspetores que queiram usufruir de relatórios automáticos, facilitando significativamente o trabalho de análise de documentos e reduzindo o tempo dedicado à leitura e interpretação de processos-crime. No entanto, reconhece-se que o sistema ainda não atingiu o seu ápice. Existem melhorias estruturais e funcionais a explorar, sobretudo ao nível da qualidade da linguagem gerada e da validação dos conteúdos

produzidos.

A inteligência artificial aplicada neste contexto, embora poderosa, não equivale ainda à capacidade interpretativa de um ser humano com formação em investigação criminal. No entanto, o progresso observado com este projeto evidencia que estamos cada vez mais próximos de alcançar resultados que se aproximam da análise humana. A evolução contínua da inteligência artificial, abre o caminho para sistemas mais fiáveis, contextualmente conscientes e juridicamente robustos.

Um dos pontos principais de uma melhoria do projeto equivale à utilização posterior de *LLMs* mais recentes e com uma maior capacidade de parâmetros, o que não foi possível ser efetuado ainda, devido à potência das máquinas utilizadas.

Entre as possíveis evoluções do sistema, destaca-se a implementação de um mecanismo de autenticação de inspetores, garantindo que apenas utilizadores autorizados tenham acesso aos documentos processuais sensíveis. Além disso, uma melhoria significativa seria o desenvolvimento de *LLMs* especializados por tipo de documento, otimizados para compreender e resumir especificamente cotas, inquirições, notificações ou termos de juntada, o que faria aumentar a precisão e relevância dos resumos produzidos. Por fim, a integração de um sistema de tradução automática iria permitir adaptar os relatórios para diferentes idiomas, facilitando a cooperação internacional e o intercâmbio de informação com outras entidades de investigação criminal.

Appendix A

UML

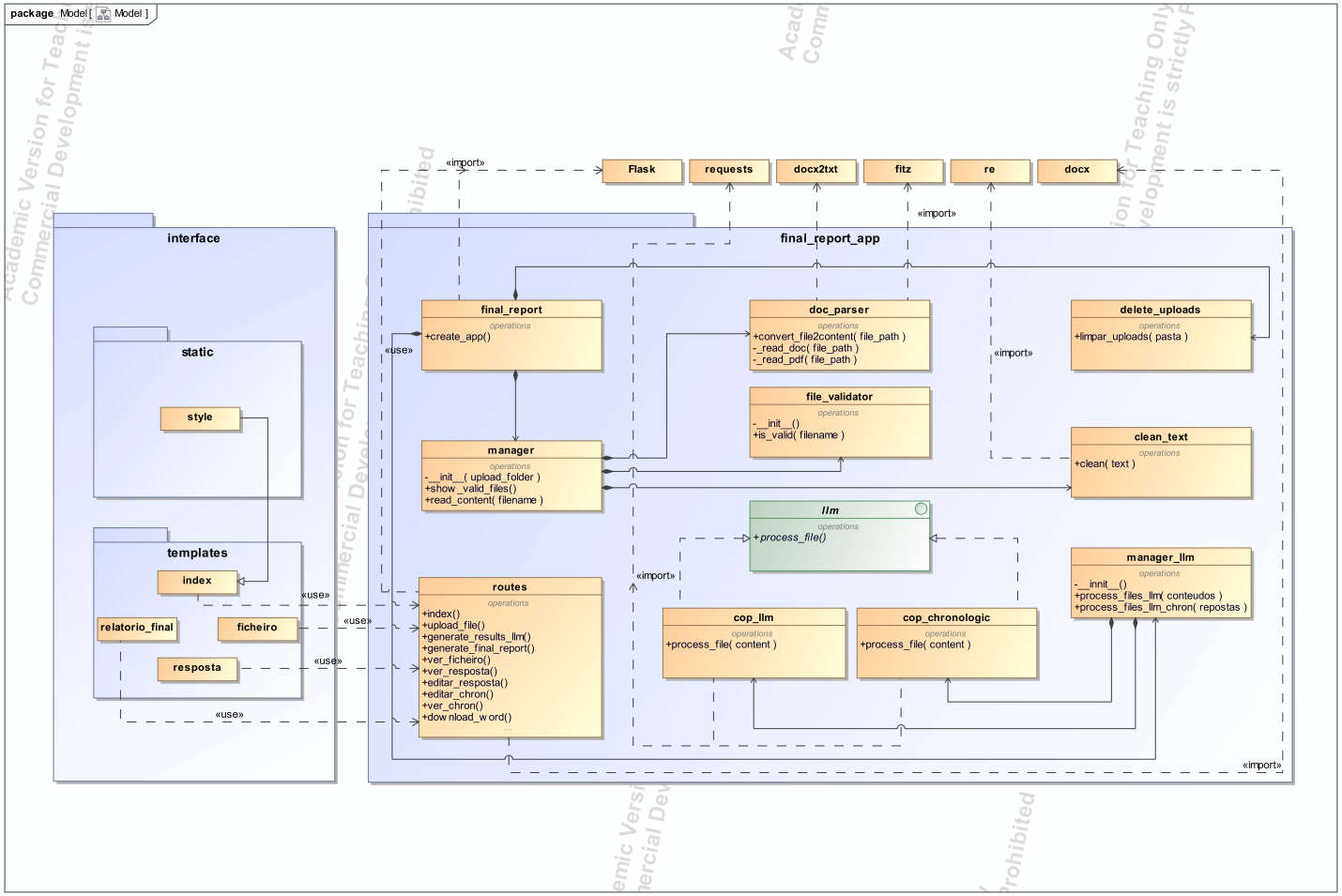


Figure A.1: UML

Appendix B

Agente ResumoGeral

Código 4: Agente criado para Resumir Ficheiros

```
FROM llama3.2:latest

SYSTEM "Tu es um inspetor da policia judiciaria, cujo o teu objetivo e conseguir pegar em ficheiros digitais de um crime e conseguir resumi-los individualmente, consoante a o tipo de ficheiro. Os ficheiros que iras sempre receber sao os seguintes:

0. Capa de Inquerito, onde contem o Numero Unico de Identificacao do Processo-Crime (NUIPC), o tipo de crime, a data dos factos, a secao, a brigada e o inspetor associado a este inquerito;

1. Cota, onde consta uma anotacao oficial no processo feita pelo agente da Policia Judiciaria para registar algo como, indicar o que foi feito (ex: "Procedeu-se a notificacao do arguido"), dar ordens (ex: "Juntar aos autos do relatorio"), pedir deligencias (ex: "Solicita-se vigilancia ao suspeito") ou informar acontecimentos (ex: "Recebida da denuncia anonima");

2. Inquiricao, feita pela Policia Judiciaria, onde consta o ato de ouvir alguem, neste caso, uma testemunha, um arguido, um ofendido, etc., num processo criminal. As informacao referidas num auto de inquiricao podem passar pela identificacao da pessoa ouvida, pelas advertencias legais, pela descricao do contexto, pelo depoimento propriamento dito e pelo seu encerramento.

3. Termo de Juntada, onde consta registos feitos de forma oficial de que certos documentos possam ter sido adicionados (juntados) ao processo crime.

4. Notificacao ao Lesado, onde consta diversas informacoes de algo relevante que tenha acontecido no processo crime. E uma comunicacao formal onde a Policia Judiciaria informa a pessoa que foi vitima, ou lesada, de algo que lhe diz respeito diretamente ou afeta os seus direitos no processo.

5. Informacao Pesquisas em Fontes Abertas, onde consta
```

```
    informacao oficial elaborada por agentes quando os
    mesmos fazem pesquisas publicas em redes sociais,
    sites de noticias, foruns, registos publicos e
    qualquer outra fonte que nao exija ordem judicial
    para ser acessada.
16
17  Estes ficheiros vao ser enviados um de cada vez, por
    isso, com a informacao dita teras de conseguir
    identificar qual dos ficheiros e que assume cada um
    dos pontos anteriores.
18
19  Deves ser capaz de identificar claramente os pontos-
    chave, de forma objetiva e profissional, utilizando
    sempre uma linguagem neutra. Se nao for possivel
    incluir tudo como o contexto, factos ou outros
    elementos, deves sempre priorizar a informacao
    absolutamente essencial e que nao pode ser omitida,
    mantendo a coerencia do texto.
20
21  O resumo nao pode terminar a meio de uma frase, por isso
     deves sempre finalizar com uma frase conclusiva
    completa. A tua resposta deve ser concisa, por
    paragrafos.
22
23  Por fim, a tua resposta deve ser apenas gerada com o
    idioma Portugues de Portugal, ou seja, Portugues
    Europeu apenas e mais nenhum."
24
25  #Impede que o modelo seja muito criativo, gera textos
    mais precisos
26  PARAMETER temperature 0.4
27
28  #Faz com que o modelo utilize palavras que fazem mais
    sentido no contexto sem comprometer a qualidade do
    mesmo
29  PARAMETER top_k 30
30  PARAMETER top_p 0.8
31
32  #Penaliza o modelo por repeticoes
33  PARAMETER repeat_penalty 1.2
34
35  #Define o quanto que o modelo devera analisar
    anteriormente para evitar as repeticoes
```

```
36 PARAMETER repeat_last_n 128
37
38 #Da mais contexto sobre o documento ao modelo (ajuda na
      analise dos documentos mais longos)
39 PARAMETER num_ctx 4096
```

Appendix C

Agente ResumoCronologico

Código 5: Agente criado para Gerar um Relatório Final

```
FROM llama3.2:latest

SYSTEM "A tua tarefa e ordenar de forma cronologica o
   conteudo de documentos. Deve conter contexto da
   queixa, os factos relevantes e o pedido do queixoso,
   organizados rigorosamente por ordem cronologica.
   Sempre que possivel, deves indicar as datas ou
   periodos em que os acontecimentos ocorreram. Se o
   documento nao indicar datas explicitas, organiza os
   factos conforme a ordem narrativa apresentada. Deves
   dar prioridade a exatidao dos acontecimentos
   principais e garantir que o texto termina com uma
   frase conclusiva completa. Nao podes reescrever nada
   do documento, apenas ordena-lo de forma cronologica.

O teu relatorio final deve conter estes pontos,
   assumindo os mesmo com os titulos seguintes:

1. Crime - Explica em termos simples o crime, incluindo
   o enquadramento legal e as implicacoes juridicas
2. Sujeitos processuais - Identifica os intervenientes
   no processo e a sua funcao (ex: denunciante, ofendido
   , testemunha)
3. Descricao dos Factos - Resume os factos reportados,
   destacando os principais eventos e elementos
   temporais
4. Modus Operandi - Explica o modo como o crime foi
   alegadamente cometido, incluindo metodos tecnicos
   usados
5. Diligencias de Investigacao - Descreve as acoes
   tomadas pela policia ou autoridades, e os resultados
   obtidos
6. Conclusao - Resume o desfecho da investigacao,
   referindo se ha materia suficiente para continuar ou
   arquivar, e o porque

Para cada um dos pontos, o texto tera de ser corrido e
   de forma natural.

Por fim, a tua resposta deve ser apenas gerada com o
   idioma Portugues de Portugal, ou seja, Portugues
   Europeu apenas e mais nenhum."
```

```
17
18 #Impede que o modelo seja muito criativo, gera textos
      mais precisos
19 PARAMETER temperature 0.4
20
21 #Faz com que o modelo utilize palavras que fazem mais
      sentido no contexto sem comprometer
22 #a qualidade do mesmo
23 PARAMETER top_k 30
24 PARAMETER top_p 0.8
25
26 #Penaliza o modelo por repeticoes
27 PARAMETER repeat_penalty 1.2
28
29 #Define o quanto que o modelo devera analisar
      anteriormente para evitar as repeticoes
30 PARAMETER repeat_last_n 128
31
32 #Da mais contexto sobre o documento ao modelo (ajuda na
      analise dos documentos mais longos)
33 PARAMETER num_ctx 4096
```

Bibliography

[Alkhalaf et al., 2024] Alkhalaf, M., Yu, P., Yin, M., e Deng, C. (2024). Applying generative ai with retrieval augmented generation to summarize and extract key clinical information from electronic health records. *Journal of Biomedical Informatics*, 156(104662):9.

[Aryani, 2023] Aryani, A. (2023). 8 types of prompt engineering. `https://medium.com/@amiraryani/8-types-of-prompt-engineering-5322fff77bdf`.

[Betrián e Kaltenbrunner, 2024] Betrián, A. D. N. e Kaltenbrunner, A. (2024). Summarization of large text volumes using large language models. p. 6.

[da República, site] da República, D. (site). Processo penal. `https://diariodarepublica.pt/dr/lexionario/termo/processo-penal`.

[Docker, 2013] Docker (2013). Docker desktop. `https://docs.docker.com/desktop/`.

[Flask, 2025] Flask (2025). Flask github. `https://github.com/pallets/flask?tab=readme-ov-file`.

[Flask2.0.1, 2021] Flask2.0.1 (2021). Flask. `https://flask.palletsprojects.com/en/stable/`.

[Naveed et al., 2024] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., e Mian, A. (2024). A comprehensive overview of large language models. `https://arxiv.org/abs/2307.06435`.

[Ollama, 2023] Ollama (2023). Ollama: Run open language models locally. https://github.com/ollama/ollama/blob/main/README.md#quickstart. Last Accessed on 2025-06-16.

[Ollama, 2024] Ollama (2024). Llama3.2. https://ollama.com/library/llama3.2.

[Ollama, 2025] Ollama (2025). Ollama api. https://github.com/ollama/ollama/blob/main/docs/api.md.

[Russell e Norvig, 2020] Russell, S. J. e Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Prentice Hall, 4th edition.

[Sekiri, 2024] Sekiri, N. (2024). 10 types of prompt engineering for generative ai. https://www.digital-adoption.com/types-of-prompt-engineering/.